

SECURECORP

Security Assessment Engagement

FINAL SECURITY ASSESSMENT REPORT

Junior Cybersecurity Consultant Capstone

Client: SecureCorp Inc.

Consultant: Neil

Assessment environment: Controlled libvirt laboratory

Final report date: 15 August 2026

Classification: CONFIDENTIAL — FOR ASSESSMENT / ACADEMIC USE

Evidence basis: direct screenshots, terminal records and working evidence retained in the SecureCorp repository.

2. Executive Summary

SecureCorp's controlled assessment identified four material weaknesses in the DVWA representation of the customer-facing web environment: brute-force authentication, SQL injection, reflected cross-site scripting and command injection. The command-injection finding is the highest-risk application weakness because successful input manipulation reached operating-system command execution in the web-server process context.

The assessment also established centralized monitoring through Wazuh. The Arch Linux physical/libvirt host operates the Wazuh Manager and Dashboard, while the Kali and Ubuntu virtual machines provide monitored endpoints. The current repository contains direct evidence of active agents, SSH authentication detection, reflected-XSS web-log detection, a SQL-injection search with no matching alert, and endpoint audit telemetry.

The defensive conclusion is therefore two-part: the application weaknesses are demonstrated by direct testing evidence, while detection effectiveness varies by attack class. SSH brute-force and reflected XSS are supported by Wazuh evidence. SQL injection is not demonstrated as detected in the retained dashboard search, and command-injection evidence demonstrates endpoint telemetry availability but does not prove that the malicious command itself generated a privileged-execution alert.

Overall posture: HIGH RISK. Immediate remediation should focus on eliminating shell-based command execution and SQL injection, strengthening authentication controls, improving XSS defenses, and validating Wazuh detections against replayed attack traffic.

3. Scope & Rules of Engagement

The engagement was restricted to the controlled SecureCorp laboratory. Testing was performed only against personally controlled virtual machines and deliberately vulnerable application components. Public systems, unrelated networks and other students' systems were out of scope.

Rule	Application
Authorized targets	Ubuntu VM hosting DVWA and associated services; controlled Kali/Arch lab systems.
Attacker environment	Kali Linux libvirt VM used for authorized security testing and analyst activity.
Central monitoring	Wazuh Manager and Dashboard hosted on the Arch Linux physical/libvirt host.
Credential testing	Restricted to deliberately created laboratory credentials and DVWA test accounts.
Network boundary	Testing remained within the controlled libvirt laboratory.
Evidence standard	Findings are supported by retained screenshots, terminal records or log evidence; unsupported event claims are not presented as fact.

The assessment follows the handbook requirement that findings must be evidence-backed and that Week 3 investigation must relate directly to the attack simulations performed during the assessment. The current repository evidence is treated as authoritative for screenshots and retained records.

4. Asset Inventory & CIA Analysis

4.1 Asset Inventory

Asset	Type	Owner / affected function	Why it matters
Customer portal / DVWA	Software	Customer-facing service	Represents the primary attack surface and handles application interactions.
Ubuntu web server	Hardware / software platform	Engineering / service operations	Hosts DVWA, Apache, MariaDB, PHP, SSH and auditd.
Customer / employee database	Data	Engineering / business operations	Contains information whose disclosure or alteration would affect confidentiality and integrity.
Employee accounts and credentials	Identity	All departments	Compromise can provide unauthorized access to business systems.
Employee laptops / Windows workstations	Endpoint	Employees	Support business operations and may hold credentials or sensitive information.
Internal company website	Software	Internal users	Provides internal information and requires integrity and availability.
Internet-facing connection	Connectivity	IT / service operations	Provides required customer access but increases attack surface.
Arch Linux Wazuh host	Infrastructure	Security monitoring	Provides centralized Wazuh management, dashboard and monitoring for the lab.

4.2 CIA Analysis

Asset	Confidentiality	Integrity	Availability
Customer portal	Customer and account data must not be disclosed.	Requests and application data must not be manipulated.	Customers must be able to use the portal.
Ubuntu web server	Logs, credentials and service data must be protected.	System and application configuration must not be altered without authorization.	Web services must remain operational.
Customer / employee database	Records must remain private.	Records must not be modified by unauthorized users.	Applications and staff must be able to access required data.
Employee accounts	Credentials must remain secret.	Account permissions and identity state must remain trustworthy.	Authorized users require reliable access.
Wazuh monitoring platform	Security telemetry should be protected from unauthorized access.	Alerts and logs must retain evidential integrity.	Monitoring must remain available for investigation.

5. Risk Assessment

Asset / area	Threat / weakness	Business impact	Risk	Primary control
Ubuntu web server	Command injection	Unauthorized OS command execution and potential server compromise.	Critical	Remove shell execution; use safe APIs, strict allowlists and least privilege.
Customer portal	SQL injection	Unauthorized disclosure or manipulation of application data.	High	Parameterized queries, server-side validation and least-privilege DB access.
Customer portal	Reflected XSS	Browser-side script execution and possible session/data exposure.	High	Context-aware output encoding, validation and CSP.
Customer portal / SSH	Brute-force authentication	Credential compromise and unauthorized access.	High	Strong passwords, rate limiting, lockout/anti-automation controls and MFA.
Employee accounts	Phishing / credential theft	Unauthorized access to business systems and data.	High	MFA, phishing-resistant authentication where practical and awareness controls.
Endpoints / server	Malware	Data loss, disruption or credential compromise.	High	Endpoint protection, patching, least privilege and centralized logging.
Business data	Insider misuse	Unauthorized disclosure or alteration.	Medium–High	Least privilege, access reviews and audit logging.
Internet-facing connection	Unauthorized scanning / exploitation	Expanded attack surface and service compromise.	High	Minimize exposed services, segment systems and monitor external activity.

6. Methodology

The assessment followed the handbook lifecycle: understand the environment, identify assets and risk, perform controlled attack simulations, collect application and endpoint telemetry, investigate through Wazuh, correlate evidence, and convert findings into prioritized remediation.

6.1 Lab and Monitoring Architecture

Role	System	Address / network	Function
Central Wazuh manager	Arch Linux physical/libvirt host	192.168.1.8	Wazuh Manager, Wazuh Dashboard and centralized monitoring.
Authorized attacker / analyst	Kali Linux libvirt VM	192.168.122.139/24	Controlled security testing and analyst activity.
Target / monitored endpoint	Ubuntu libvirt VM	192.168.122.138/24	DVWA target; Apache, MariaDB, PHP, SSH and auditd; Wazuh agent.
Lab gateway	Libvirt network	192.168.122.1	Gateway for the VM network.

SecureCorp — Week 1 Lab Architecture

Authorized self-contained assessment environment

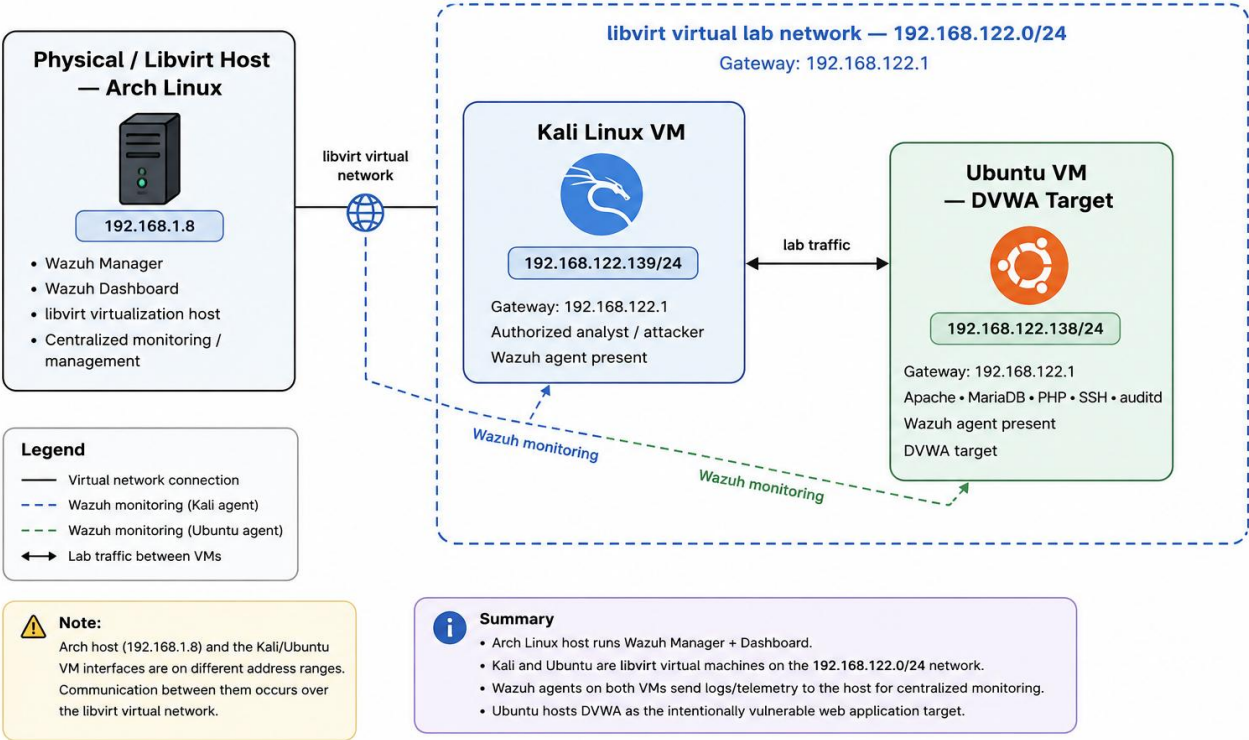


Figure 6-1. Repository network diagram showing the Arch Linux host, Kali VM, Ubuntu/DVWA target and Wazuh monitoring paths.

6.2 Wazuh Deployment

The Arch Linux host provides centralized Wazuh management and dashboard access. The repository's current Wazuh agent view shows two active agents: the Arch host and the Ubuntu endpoint. The Ubuntu endpoint also retains direct evidence of the Wazuh agent running.

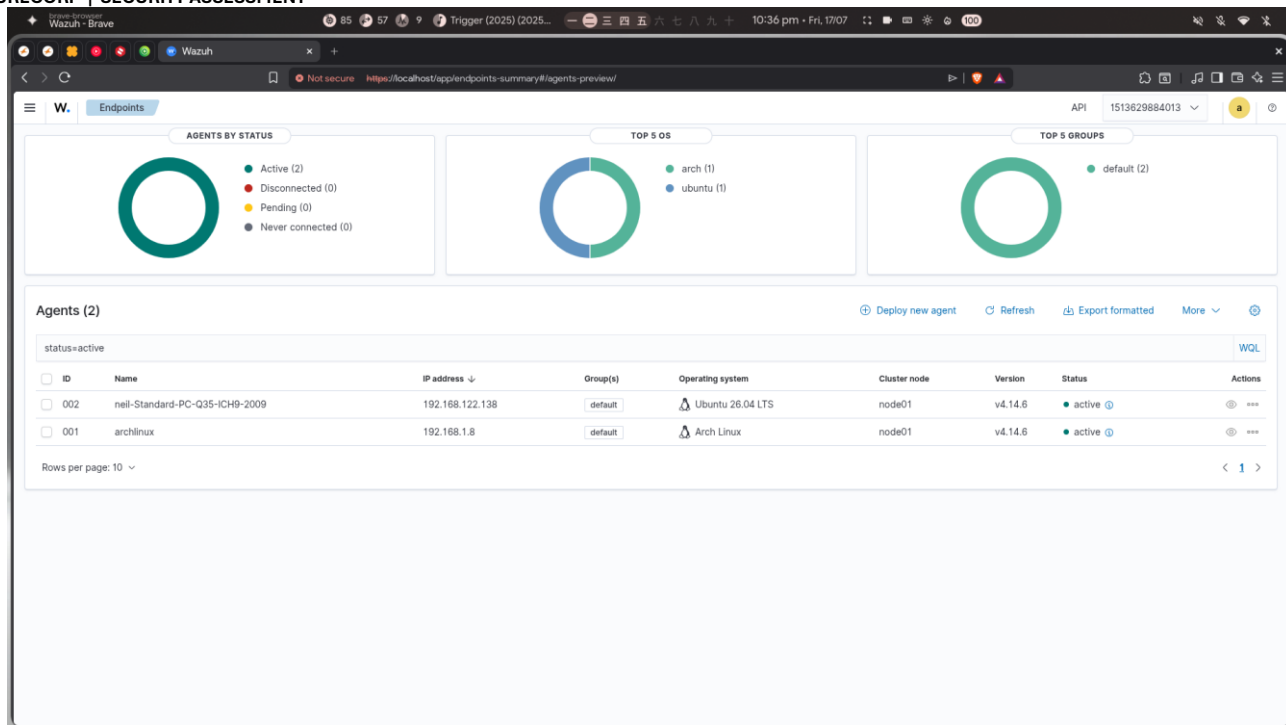


Figure 6-2. Direct Wazuh screenshot showing both documented agents active, including the Arch Linux host and Ubuntu endpoint.

6.3 Assessment Workflow

- Validate host identities, IP addressing, services, DVWA availability and Wazuh connectivity.
- Execute the four required controlled attack simulations against the DVWA target and controlled SSH service.
- Record direct application results and timestamps where available.
- Review Wazuh authentication, web-access-log and endpoint/audit telemetry.
- Separate exploitation evidence from detection evidence; a successful attack does not automatically constitute a confirmed SIEM alert.
- Prioritize remediation according to technical impact and business consequence.

7. Attack Simulation Findings

The four required simulations are summarized below. Dates and times are stated only where supported by retained evidence. Where a direct screenshot lacks a timestamp, the corresponding retained terminal record is used and identified as such.

Attack	Evidence timestamp	Result	Detection status	Risk
Brute-force authentication	13 Aug 2026, 07:57:06 -0400 (access log evidence)	Controlled repeated authentication activity and successful lab authentication were demonstrated.	Confirmed Wazuh SSH detection; rules 5710/related SSH telemetry retained.	High
SQL injection	13 Aug 2026, 08:02:02 -0400 (access log evidence)	Boolean manipulation returned multiple records, demonstrating query alteration.	Not confirmed. Retained Wazuh search returned no matching results.	High
Reflected XSS	13 Aug 2026, 08:11:44 -0400 (access log evidence)	Injected script executed in the browser and generated the expected alert behaviour.	Confirmed Wazuh web-access-log evidence; rule 31106 shown in retained event.	High
Command injection	24 Jun 2026, 12:13:19–12:19:52 IST (terminal record)	Appended shell commands returned uid=33(www-data), whoami and working-directory output.	Endpoint audit telemetry is present; malicious privileged-execution alert not proven.	Critical

7.1 Brute Force

The DVWA brute-force module and controlled SSH authentication path were assessed. The retained evidence shows the vulnerable login area and access-log activity. The working attack record states that a valid laboratory credential was obtained and authenticated successfully.



[Home](#)
[Instructions](#)
[Setup / Reset DB](#)
[Brute Force](#)
[Command Injection](#)
[CSRF](#)
[File Inclusion](#)
[File Upload](#)
[Insecure CAPTCHA](#)
[SQL Injection](#)
[SQL Injection \(Blind\)](#)
[Weak Session IDs](#)
[XSS \(DOM\)](#)
[XSS \(Reflected\)](#)
[XSS \(Stored\)](#)
[CSP Bypass](#)
[JavaScript](#)
[Authorisation Bypass](#)
[Open HTTP Redirect](#)
[Cryptography](#)
[API](#)

[DVWA Security](#)
[PHP Info](#)
[About](#)

[Logout](#)

Vulnerability: Brute Force

Login

Username:

Password:

Welcome to the password protected area admin



More Information

- https://owasp.org/www-community/attacks/Brute_force_attack
- <https://www.symantec.com/connect/articles/password-crackers-ensuring-security-your-password>
- <https://www.golinuxcloud.com/brute-force-attack-web-forms>

Username: admin
 Security Level: low
 Location:

[View Source](#) [View Help](#)

Figure 7-1. Direct DVWA brute-force evidence.

```

kali@kali:~$ sudo grep 'POST /vulnerabilities/brute' /var/log/dvwa/access.log | tail -10
127.0.0.1 - - [13/Aug/2026:07:46:39 -0400] "POST /vulnerabilities/brute/ HTTP/1.1" 302 5 "-" "curl/8.21.0"
127.0.0.1 - - [13/Aug/2026:07:47:12 -0400] "POST /vulnerabilities/brute/ HTTP/1.1" 302 5 "-" "curl/8.21.0"
127.0.0.1 - - [13/Aug/2026:07:47:12 -0400] "POST /vulnerabilities/brute/index.php HTTP/1.1" 200 4585 "-" "curl/8.21.0"
127.0.0.1 - - [13/Aug/2026:07:57:06 -0400] "POST /vulnerabilities/brute/ HTTP/1.1" 302 5 "-" "curl/8.21.0"
127.0.0.1 - - [13/Aug/2026:07:57:06 -0400] "POST /vulnerabilities/brute/index.php HTTP/1.1" 200 4585 "-" "curl/8.21.0"

kali@kali:~$ sudo grep '07:57:06.*POST /vulnerabilities/brute' /var/log/dvwa/access.log
127.0.0.1 - - [13/Aug/2026:07:57:06 -0400] "POST /vulnerabilities/brute/ HTTP/1.1" 302 5 "-" "curl/8.21.0"
127.0.0.1 - - [13/Aug/2026:07:57:06 -0400] "POST /vulnerabilities/brute/index.php HTTP/1.1" 200 4585 "-" "curl/8.21.0"

```

Figure 7-2. Direct access-log evidence with the retained brute-force request timestamps.

7.2 SQL Injection

The DVWA SQL Injection module accepted boolean-manipulation input and returned multiple records. This demonstrates unsafe query handling and a high confidentiality/integrity risk.

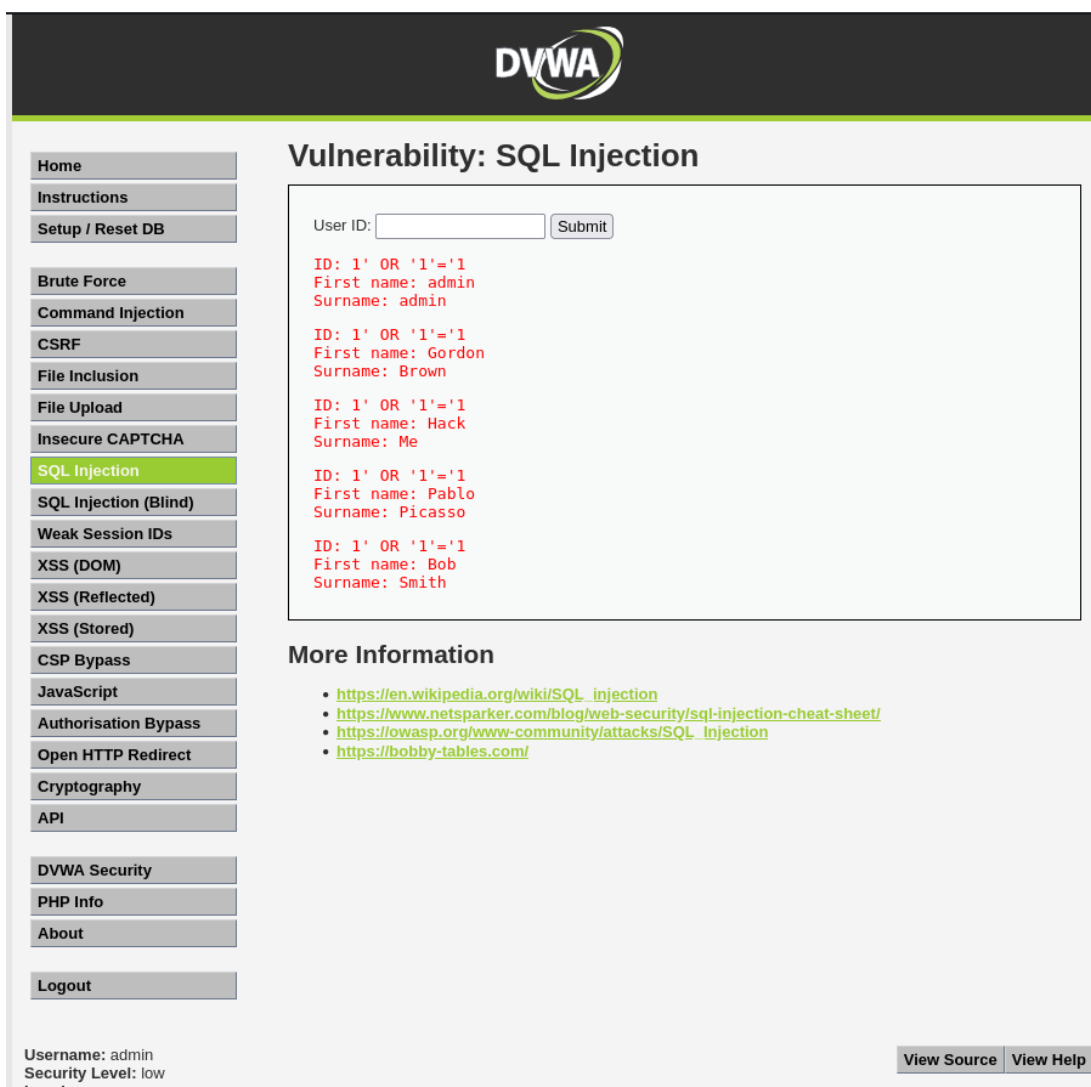


Figure 7-3. Direct DVWA SQL injection result.

```

[kali@kali:~]$ sudo grep "08:02:02.vulnerabilities/sqli" /var/log/dmwa/access.log
127.0.0.1 - - [13/Aug/2026:08:02:02 +0400] "GET /vulnerabilities/sqli/?id=1x27+0R+K271x27x3Dx2715Submit=Submit HTTP/1.1" 200 1553 "http://127.0.0.1:42001/vulnerabilities/sqli/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"

```

Figure 7-4. Direct access-log evidence for the SQL injection request.

7.3 Reflected XSS

The reflected-XSS module rendered attacker-controlled script input in the browser. The retained Wazuh document shows the corresponding web-access-log event and rule 31106.

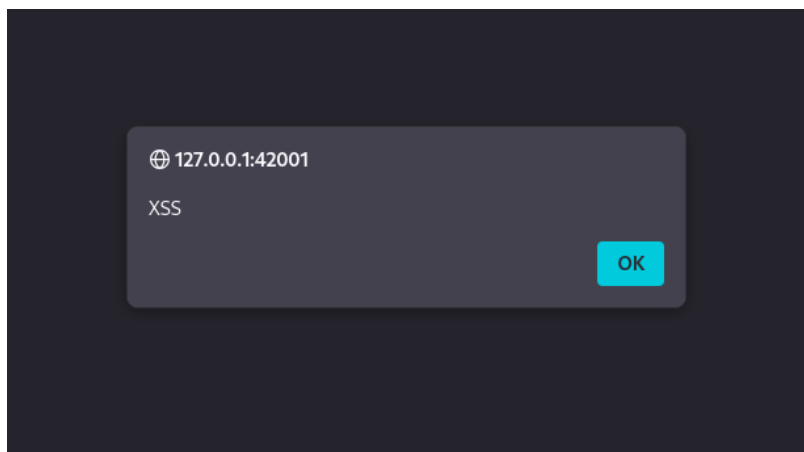


Figure 7-5. Direct browser evidence of reflected XSS execution.

```
(kali@kali) ~$ sudo grep '08:11:44.*xss.*' /var/log/dvwa/access.log
127.0.0.1 - - [13/Aug/2026:08:11:44 -0400] "GET /vulnerabilities/xss_r/?name=%3Cscript%3Ealert%28%27XSS%27%29%3C%2Fscript%3E HTTP/1.1" 200 1512 "http://127.0.0.1:42001/vulnerabilities/xss_r/?name" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
```

Figure 7-6. Direct access-log evidence with the retained XSS timestamp.

7.4 Command Injection

The command-injection module accepted appended shell commands. The retained terminal record demonstrates execution as www-data and exposes the application's working directory. This is the highest-risk application finding because the attack crosses from the web layer into operating-system execution.



Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

Authorisation Bypass

Open HTTP Redirect

Cryptography

API

DVWA Security

PHP Info

About

Logout

Vulnerability: Command Injection

Ping a device

Enter an IP address:

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.253 ms  
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.030 ms  
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.024 ms  
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.025 ms  
  
--- 127.0.0.1 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3074ms  
rtt min/avg/max/mdev = 0.024/0.083/0.253/0.098 ms  
_dvwa
```

More Information

- <https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
- <http://www.ss64.com/bash/>
- <http://www.ss64.com/nt/>
- https://owasp.org/www-community/attacks/Command_Injection

Username: admin
Security Level: low

Figure 7-7. Direct DVWA command-injection execution evidence.

8. Detection Notes & Incident Timeline

8.1 Detection Notes

Attack	Wazuh / telemetry mapping	Evidence conclusion
Brute-force authentication	SSH/authentication rules including 5710 and supporting 5720/5716 context.	Confirmed detection evidence retained.
SQL injection	Web-access-log investigation; retained dashboard search for the SQLi path.	No matching result in retained dashboard evidence; detection not confirmed.
Reflected XSS	Web-access-log rule 31106; document details show /vulnerabilities/xss_r/ and HTTP 200.	Confirmed detection evidence retained.
Command injection	Web-layer investigation plus auditd endpoint telemetry; 80792 is relevant when its documented privileged condition is present.	Telemetry is present, but retained evidence does not prove a malicious privileged-execution alert.

8.2 Incident Timeline

Evidence time	Activity	Source	Assessment
13 Aug 2026 07:57:06 -0400	Brute-force web authentication activity	DVWA access log screenshot	Attack activity recorded.
13 Aug 2026 08:02:02 -0400	SQL injection request	DVWA access log screenshot	Attack request recorded; Wazuh search did not return a matching result.
13 Aug 2026 08:11:44 -0400	Reflected XSS request	DVWA access log + Wazuh document	Attack and web-log detection corroborated.
13 Aug 2026 08:18:15–08:18:21	SSH authentication failures / invalid-user attempts	Wazuh SSH events	Wazuh rule 5710 evidence retained.
13 Aug 2026 13:30:09	Endpoint audit search command executed through sudo	Wazuh endpoint document	Telemetry collection is functioning; this is analyst activity, not malicious command proof.

8.3 Investigation Conclusion

The monitoring architecture provides useful coverage across authentication, web access logs and endpoint telemetry. The evidence demonstrates confirmed Wazuh visibility for SSH brute-force and reflected XSS, while SQL injection and command-injection detection require further rule validation. The distinction is important: application compromise findings remain valid even when the defensive detection control does not fire.

Document Details

[View surrounding documents](#)
[View single document](#)

[Table](#)

JSON

@index_name	wazuh-alerts
@src_ip	127.0.0.1
_index	wazuh-alerts-4.x-v1-2026.08.13-000003
agent.id	001
agent.ip	10.0.2.15
agent.name	dvwa
cluster.name	0sxx3i088eng
cluster.node	wazuh-manager-master-0
data.srcip	127.0.0.1
data.srcuser	wronguser
decoder.name	sshd
decoder.parent	sshd
full_log	Aug 13 12:18:18 kali sshd-session[118555]: Failed password for invalid user wronguser from 127.0.0.1 port 59072 ssh2
id	1786623501.4938
input.type	log
location	journald
manager.name	wazuh-manager-master-0
predecoder.hostname	kali
predecoder.program_name	sshd-session
predecoder.timestamp	Aug 13 12:18:18
rule.description	sshd: Attempt to login using a non-existent user
rule.firedtimes	2

Figure 8-1. Wazuh document showing SSH invalid-user authentication activity.

Document Details [View surrounding documents](#) [View single document](#) ✕

Table JSON

t @index_name	wazuh-alerts
t @src_ip	127.0.0.1
t _index	wazuh-alerts-4.x-v1-2026.08.13-000003
t agent.id	001
t agent.ip	10.0.2.15
t agent.name	dvwa
t cluster.name	0sxcg3i088eng
t cluster.node	wazuh-manager-master-0
t data.id	200
t data.protocol	GET
t data.srcip	127.0.0.1
t data.url	/vulnerabilities/xss_r/?name=%3Cscript%3Ealert%28%27XSS%27%29%3C%2Fscript%3E
t decoder.name	web-accesslog
t full_log	127.0.0.1 - - [13/Aug/2026:08:11:44 -0400] "GET /vulnerabilities/xss_r/?name=%3Cscript%3Ealert%28%27XSS%27%29%3C%2Fscript%3E HTTP/1.1" 200 1512 "http://127.0.0.1:42001/vulnerabilities/xss_r/?name="Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
t id	1786623104.5941
t input.type	log
t location	/var/log/dvwa/access.log
t manager.name	wazuh-manager-master-0
t rule.description	A web attack returned code 200 (success).
# rule.firedtimes	1
t rule.gdpr	IV_35.7.d

Figure 8-2. Wazuh document showing the reflected-XSS web-access event and rule 31106.

W. Threat Hunting dvwa API 1 W ?

Dashboard Events dvwa (001)

agent.name:dvwa AND *1%27*OR*%271%27%3D%271*

DQL Last 120 minutes Show dates Refresh

cluster.name: 0sxcg3i088eng agent.id: 001 agent.name: dvwa Add filter

No results match your search criteria

Figure 8-3. Retained Wazuh search showing no results for the SQL injection query.

Document Details		View surrounding documents	View single document	✕
Table JSON				
t	@index_name	wazuh-alerts		
t	_index	wazuh-alerts-4.x-v1-2026.08.13-000003		
t	agent.id	001		
t	agent.ip	10.0.2.15		
t	agent.name	dvwa		
t	cluster.name	0sxc3i088eng		
t	cluster.node	wazuh-manager-master-0		
t	data.command	/usr/sbin/ausearch -m EXECVE --start recent		
t	data.dstuser	root		
t	data.pwd	/home/kali		
t	data.srcuser	kali		
t	data.tty	pts/1		
t	decoder.ftscoment	First time user executed the sudo command		
t	decoder.name	sudo		
t	decoder.parent	sudo		
t	full_log	Aug 13 13:30:09 kali sudo[156953]: kali : TTY=pts/1 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/sbin/ausearch -m EXECVE --start recent		
t	id	1786627811.61624		
t	input.type	log		
t	location	journald		
t	manager.name	wazuh-manager-master-0		
t	predecoder.hostname	kali		
t	predecoder.program_name	sudo		

Figure 8-4. Wazuh endpoint telemetry showing audit/ausearch activity; this does not prove malicious privileged execution.

9. Recommendations

Priority	Area	Recommendation	Success measure
Critical	OS command execution	Remove shell execution from web input. Use safe APIs, strict allowlists, validation and least-privilege service accounts.	Command-injection test no longer produces OS command output.
Critical	SQL injection	Use parameterized queries, server-side validation and least-privilege database permissions.	SQLi payloads fail safely and do not alter query logic.
High	Authentication	Use strong unique credentials, rate limiting, lockout/anti-automation controls and MFA for sensitive access.	Repeated authentication attempts are throttled and protected accounts resist credential guessing.
High	XSS	Apply context-aware output encoding, input validation and an appropriate Content Security Policy.	Injected script is rendered inert and CSP violations are controlled.
High	Wazuh detection engineering	Validate and tune rules against replayed SQLi, XSS, command-injection and SSH attack traffic.	Each required attack produces a predictable, actionable detection where intended.
High	Privileged access	Minimize sudo/su access and retain privileged-command auditing.	Unexpected privileged execution is attributable to a user/process and investigation can reconstruct the sequence.
High	Patch management	Keep the OS, web server, PHP, database and application components supported and patched.	Critical security updates are applied within an agreed maintenance window.
Medium-High	Network exposure	Expose only required services and restrict management access to trusted paths.	External attack surface is minimized and management interfaces are not publicly reachable.
Medium-High	Backup and recovery	Maintain protected backups and periodically test restoration.	Recovery objectives are met during a controlled restore test.

Recommended implementation order: eliminate command execution and SQL injection first; harden authentication and XSS controls next; then validate Wazuh coverage and privileged-activity monitoring against repeatable test cases.

10. Appendices

Appendix A — Evidence Repository

The repository is the authoritative evidence location. The final report embeds selected direct screenshots; the complete evidence set remains available in the project repository.

Evidence	Repository location
Network diagram	SecureCorp/workingevidence/week1/03_Network_Diagram/final_network_diagram.png
Arch/Wazuh screenshots	SecureCorp/Screenshots/arch-host/
Kali attack screenshots	SecureCorp/Screenshots/kali/
Ubuntu service / agent screenshots	SecureCorp/Screenshots/ubuntu/
Attack logs	SecureCorp/workingevidence/week2/03_Attack_Logs/
Terminal transcripts	SecureCorp/workingevidence/week2/02_Terminal_Transcripts/
Detection notes / Wazuh investigation	SecureCorp/workingevidence/week3/
Recommendations / final working material	SecureCorp/workingevidence/week4/

Repository: github.com/Ameyrnx/aenexz/tree/master/SecureCorp

Appendix B — Environment Network Diagram

SecureCorp — Week 1 Lab Architecture

Authorized self-contained assessment environment

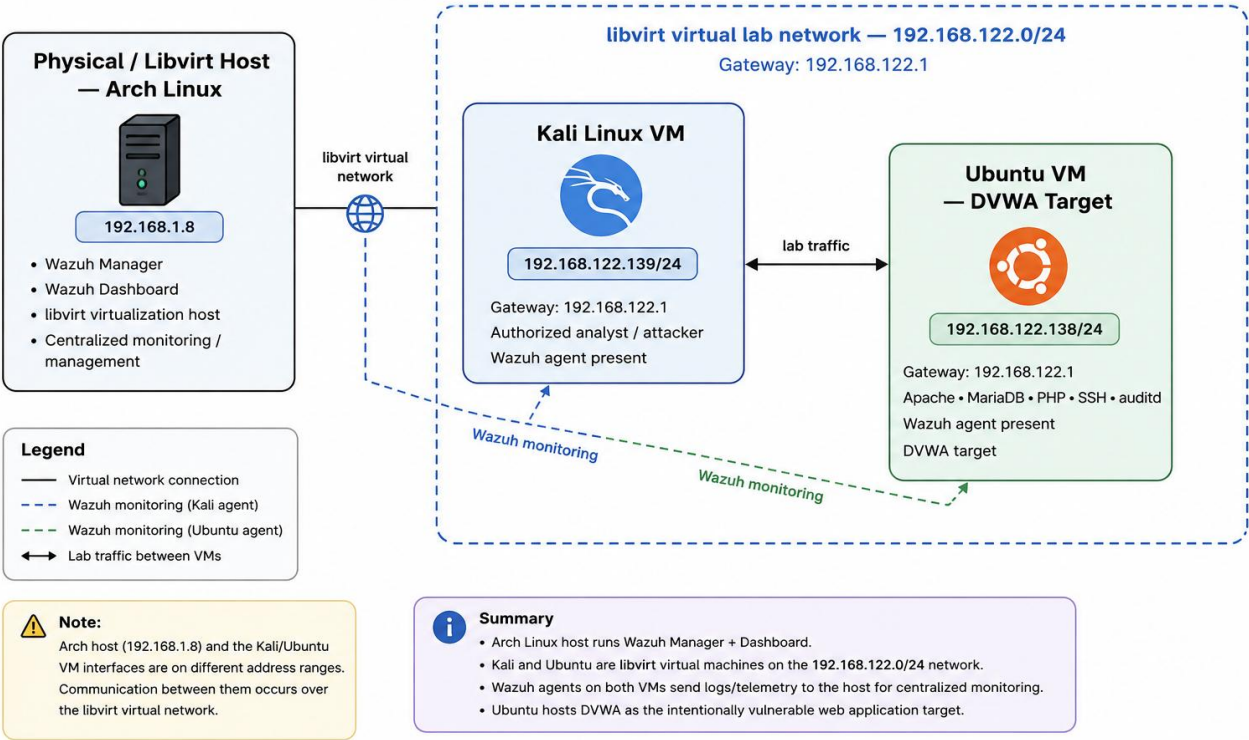


Figure B-1. Final repository network diagram. Source retained in the project evidence set.

Appendix C — Evidential Screenshots

The following figures are the key direct screenshots required to demonstrate the assessment. Full supporting evidence remains in the repository.

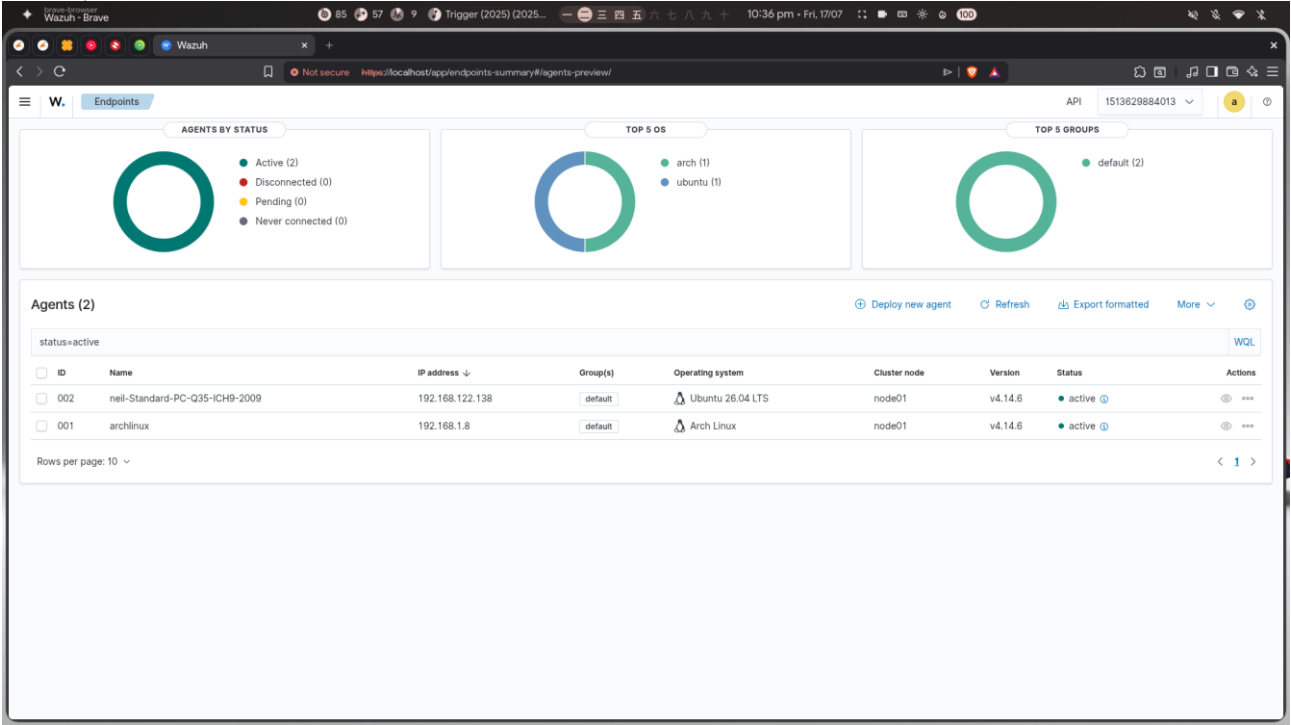


Figure C-1. Wazuh agents active.

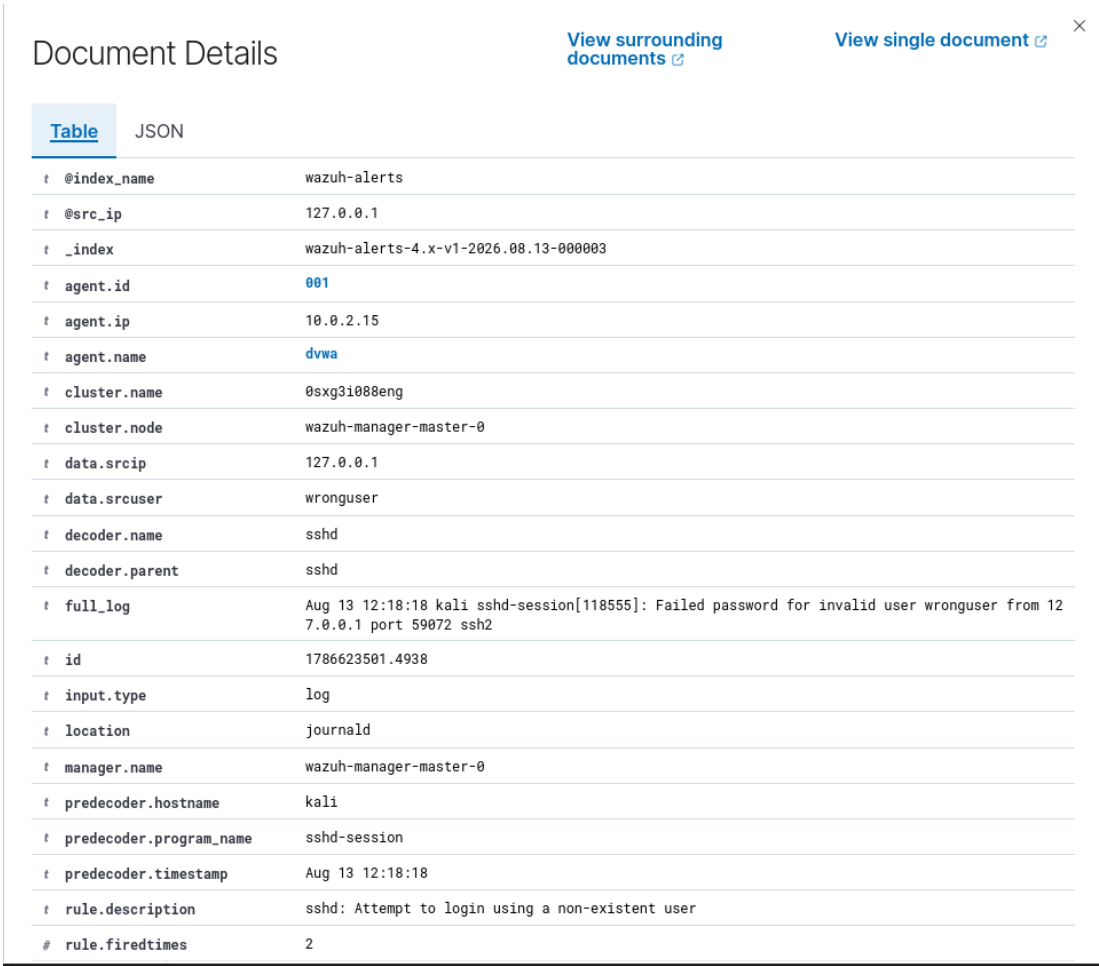


Figure C-2. SSH authentication detection.

W. Threat Hunting dwwa API 1 W

Dashboard Events dwwa (001)

agent.name:dwwa AND "1%27+OR+%271%27%3D%271"

DQL Last 120 minutes Show dates Refresh

cluster.name: 0sxc3i088eng agent.id: 001 agent.name: dwwa Add filter

No results match your search criteria

Figure C-3. SQL injection detection gap.

API 1 W

Document Details View surrounding documents View single document

Table JSON

@index_name	wazuh-alerts
@src_ip	127.0.0.1
_index	wazuh-alerts-4.x-v1-2026.08.13-000003
agent.id	001
agent.ip	10.0.2.15
agent.name	dwwa
cluster.name	0sxc3i088eng
cluster.node	wazuh-manager-master-0
data.id	200
data.protocol	GET
data.srcip	127.0.0.1
data.url	/vulnerabilities/xss_r/?name=%3Cscript%3Ealert%28%27XSS%27%29%3C%2Fscript%3E
decoder.name	web-accesslog
full_log	127.0.0.1 - - [13/Aug/2026:08:11:44 -0400] "GET /vulnerabilities/xss_r/?name=%3Cscript%3Ealert%28%27XSS%27%29%3C%2Fscript%3E HTTP/1.1" 200 1512 "http://127.0.0.1:42001/vulnerabilities/xs_s_r/?name" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"
id	1786623104.5941
input.type	log
location	/var/log/dwwa/access.log
manager.name	wazuh-manager-master-0
rule.description	A web attack returned code 200 (success).
rule.firedtimes	1
rule.gdpr	IV_35.7.d

Figure C-4. Reflected-XSS detection.

Document Details [View surrounding documents](#) [View single document](#)

Table JSON

@index_name	wazuh-alerts
_index	wazuh-alerts-4.x-v1-2026.08.13-000003
agent.id	001
agent.ip	10.0.2.15
agent.name	dvwa
cluster.name	0sxcg3i088eng
cluster.node	wazuh-manager-master-0
data.command	/usr/sbin/auresearch -m EXECVE --start recent
data.dstuser	root
data.pwd	/home/kali
data.srcuser	kali
data.tty	pts/1
decoder.ftscomment	First time user executed the sudo command
decoder.name	sudo
decoder.parent	sudo
full_log	Aug 13 13:30:09 kali sudo[156953]: kali : TTY=pts/1 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/sbin/auresearch -m EXECVE --start recent
id	1786627811.61624
input.type	log
location	journald
manager.name	wazuh-manager-master-0
predecoder.hostname	kali
predecoder.program_name	sudo

Figure C-5. Endpoint audit telemetry.

Appendix D — Supporting Logs and Evidence Notes

- The direct application screenshots demonstrate the four required attack outcomes.
- The retained access-log screenshots provide evidence timestamps for brute force, SQL injection and XSS.
- The command-injection terminal transcript records the controlled test window and the returned OS-level command output.
- Wazuh evidence is reported separately from exploitation evidence.
- The SQL injection dashboard screenshot is retained as evidence of a detection gap, not as evidence of a successful alert.
- The endpoint audit screenshot demonstrates telemetry availability but is not interpreted as proof of malicious privileged execution.
- Historical or unsupported system-specific values are not presented as verified current facts.

Appendix E — Submission Checklist

- ☐ Network Diagram / Application Map — included.
- ☐ Asset Inventory — included.
- ☐ CIA Analysis — included.
- ☐ Risk Assessment — included.
- ☐ Wazuh dashboard screenshot with active agents — included.
- ☐ Attack Log covering brute force, SQL injection, XSS and command injection with evidence/timestamps where supported — included.
- ☐ Detection notes for all four attack classes — included.
- ☐ Incident timeline with retained evidence times — included.
- ☐ Security Recommendations — included.
- ☐ Executive Summary — included.
- ☐ Evidence files organized in the SecureCorp repository — referenced.